IUT Robert Schuman
Institut universitaire de technologie
Université de Strasbourg

P11 – Initiation à la programmation

-

Les fichiers texte séquentiels

-

Les structures

Hervé L'helguen

Plan

1. Introduction sur les fichiers texte séquentiels

Introduction fichiers texte séquentiels

Un fichier est une zone de mémoire stockée sur un support durable (disque dur magnétique, disque SSD, clé USB…) permettant de conserver des informations entre deux exécutions d'un programme. On distingue deux grandes familles de fichiers :

- Les fichiers binaires
- Les fichiers texte

Un **fichier binaire** contient des données codées non directement lisibles par un humain. C'est le cas, par exemple, des fichiers images (.png ou .jpg), des fichiers exécutables (.exe) ou encore des fichiers son (.mp3).

Un **fichier texte** est composé uniquement de caractères lisibles et peut être ouvert par un éditeur de texte basique. Ces fichiers sont utilisés pour sauvegarder des données simples et non confidentielles (listes, paramètres, journaux, …) et servent également à l'échange de données entre plusieurs programmes en respectant des formats standards comme CSV, JSON ou XML.

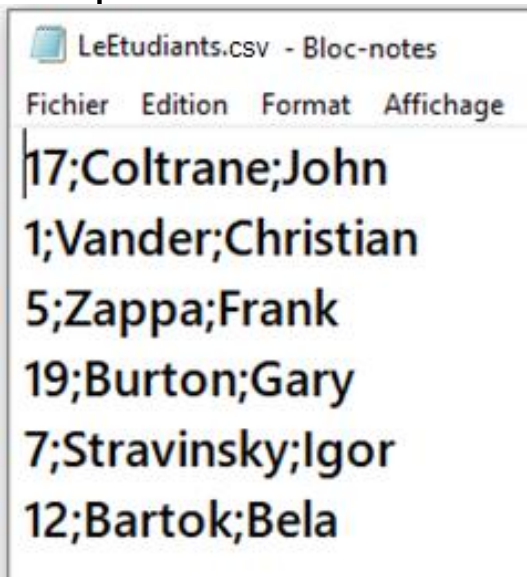
Introduction fichiers texte séquentiels



Le format CSV (Comma-Separated Values) est un format de texte pur qui contient des données multiples toutes séparées par des virgules (ou point-virgules). L'extension utilisés est "**.csv**".

- Il a l'avantage d'être lu par beaucoup de logiciels en natif et plus particulièrement par les tableurs (lignes / colonnes comme Excel).
- Les SGBD permettent d'importer des fichiers CSV pour remplir des tables.
- Les langages de programmation possèdent des bibliothèques CSV.

Par exemple : "lesEtudiants.csv" :



Introduction fichiers texte séquentiels



Le format XML (eXtensible Markup Language) est une norme qui sert à définir une structure de manière arborescente. XML est un langage à "balises", mais contrairement à HTML il n'est pas sémantiquement figé car Il permet de définir ses propres balises, ce qui le rend adaptable et donc à même de stocker tous types d'informations. Un document XML ne contient que du texte pur, l'extension utilisée pour ces fichiers est **".xml".**

- Lu en natif dans les navigateurs
- Le XML sert à structurer et échanger des données
- Domaines très divers : sites et services web, dessins vectoriels, généalogie ou serveurs vocaux.

Par exemple : "cd.xml" :

```
<?xml version="1.0" encoding="UTF-8"?>
<cd>
    <titre>Joe's Garage</titre>
    <photo>joesgarage.jpg</photo>
    <interprete>Frank Zappa</interprete>
    <genre>Rock</genre>
    <piste>
        <numero>1</numero>
        <titre>Crew Slut</titre>
        <duree>3:07</duree>
    </piste>
    <piste>
        <numero>2</numero>
        <titre>Entrance Of The Central Scrutinizer</titre>
        <duree>0:59</duree>
    </piste>
</cd>
```

Référence à la version de xml et à l'encodage

Introduction fichiers texte séquentiels



Le format JSON (JavaScript Object Notation) est un format de donnée d'échange en texte pur. L'extension utilisée est "**.json**". Un fichier JSON permet de stocker des données structurées à base de couples **clé / valeur**.

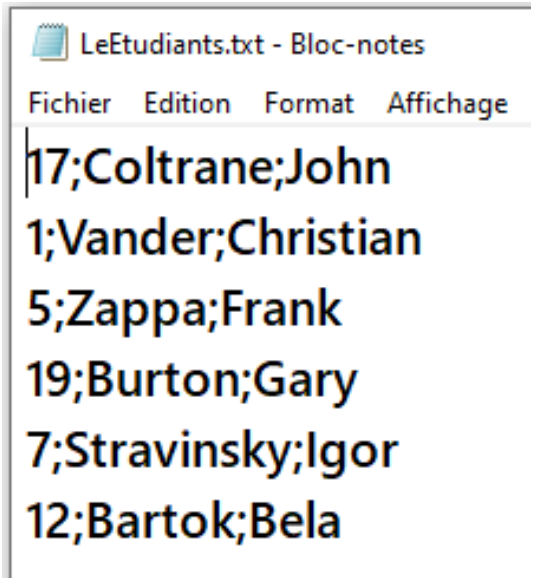
- Lu en natif dans les navigateurs.
- Les langages de programmation orientés objets possèdent des bibliothèques JSON.

Par exemple : "cd.json" :

```
{
        "titre": "Joe's Garage",
        "photo": "joesgarage.jpg",
        "interprete": "Frank Zappa",
        "genre": "Rock",
        "pistes": [
         {
                "numero": "1",
                "titre": "Crew Slut",
                "duree": "2:51"
         },
         {
                "numero": "2",
                "titre": "Entrance Of The Central Scrutinizer",
                "duree": "0:59"
         }
        ]
}
```

Introduction fichiers texte séquentiels

Dans le cadre de cette initiation en P11 nous allons uniquement traiter les **fichiers texte (donc potentiellement des fichiers CSV, XML ou JSON)** vous aurez l'occasion de manipuler des fichiers binaires en cours Système. Voici un extrait du contenu du fichier "**lesEtudiants.txt**" :



Ce fichier texte est qualifié de **séquentiel** car les données sont enregistrées les unes à la suite des autres, ici un enregistrement par ligne.

Plan

1. Introduction sur les fichiers séquentiels
2. Mode d'exploitation d'un fichier texte séquentiel

Modes d'exploitation d'un fichier texte séquentiel

Les fichiers texte séquentiels peuvent être exploités de 3 façons qui vont dépendre du traitement que l'on veut faire avec le fichier :

- Mode Lecture //Consultation du contenu du fichier
- Mode Ajout //Ajout d'enregistrements en fin de fichier
- Mode Écriture //On écrase tout et on ajoute des enregistrements

Plan

1. Introduction sur les fichiers séquentiels
2. Mode d'exploitation d'un fichier séquentiel
3. Déclaration et ouverture d'un fichier

La déclaration et ouverture d'un fichier

Je veux déclarer le fichier lesEtudiants dans un algorithme :

```
Algorithme testFichier
    Déclarations
        Variables
            lesEtudiants : fichier séquentiel
Début
  Ouvrir lesEtudiants associé à "lesEtudiants.txt" en Lecture
```

Ou en Ajout
Ou en Ecriture

Nom logique
associé au flux
d'entrée/sortie
entre le programme
et le fichier réel

nom physique
associé au fichier réel
sur le support durable

Plan

1. Introduction sur les fichiers séquentiels
2. Mode d'exploitation d'un fichier séquentiel
3. Déclaration et ouverture d'un fichier
4. Lire le contenu du fichier

Lire le contenu du fichier

Les bonnes pratiques : Commençons par vérifier que le fichier existe bien (pour éviter un plantage) puis ouvrons-le. Cette action consiste à ouvrir un flux de données entre le programme et le fichier physique. Ce flux sera fermé après le traitement.

```
Algorithme testFichier
    Déclarations
        Variables
            lesEtudiants : fichier séquentiel
            ligne : chaîne de caractères
Début
    Si Existe("lesEtudiants.txt") Alors  //Je teste l'existence du fichier
        Ouvrir lesEtudiants associé à "lesEtudiants.txt" en Lecture
        //Ici diverses actions avec le fichier…
        Fermer lesEtudiants  //Je  ferme le flux d'entrée/sortie !
    Sinon
        Afficher "Le fichier n'a pas pu être ouvert"
    FinSi
```

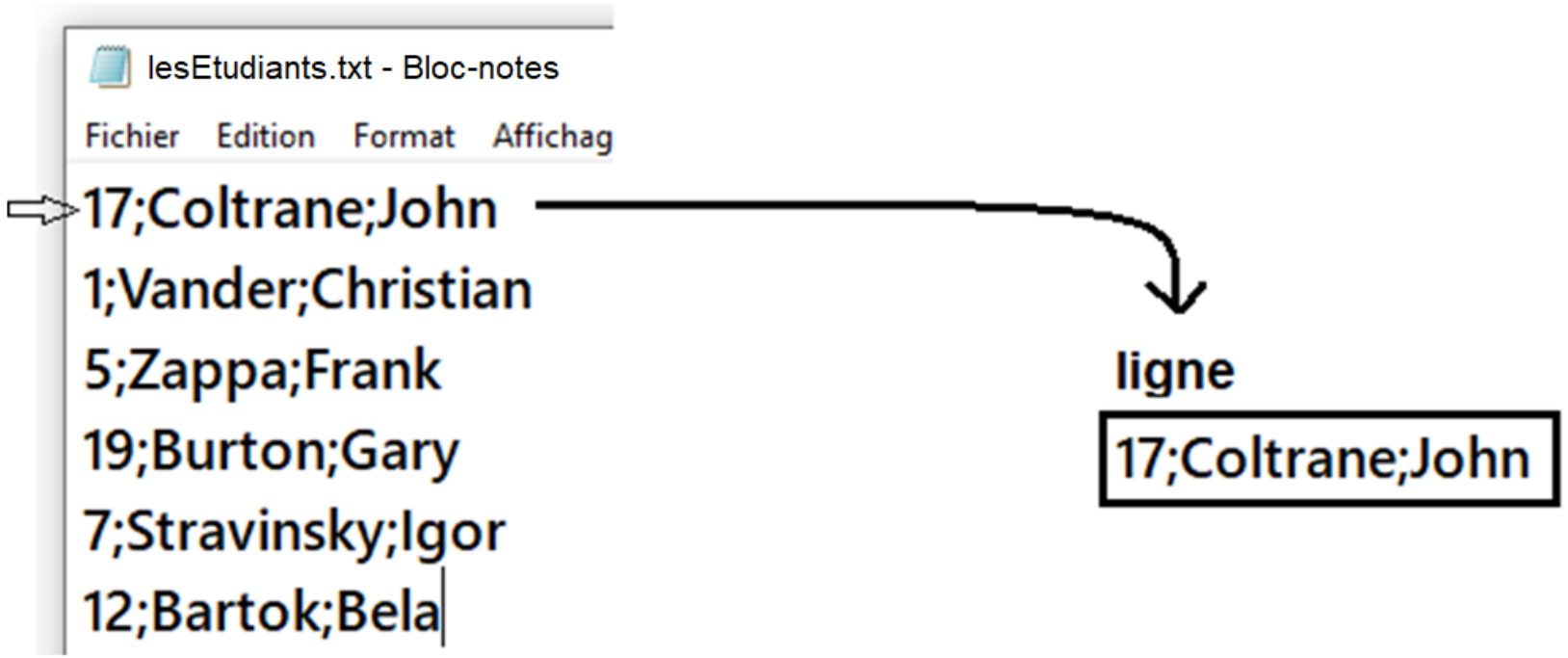
Lire le contenu du fichier

L'accès aux enregistrements se fait de façon séquentielle, c'est à dire qu'à la première lecture du fichier on accède au 1er enregistrement, à la deuxième lecture au deuxième enregistrement et ainsi de suite jusqu'à la fin du fichier. Il n'est donc pas possible d'accéder directement au 33ème enregistrement, il faut lire les 32 autres avant !

```
Début
    Si Existe("lesEtudiants.txt") Alors
        Ouvrir lesEtudiants associé à "lesEtudiants.txt" en Lecture
        ligne = Lire lesEtudiants        //1ère lecture
        Afficher ligne
        ligne = Lire lesEtudiants        //2ème lecture
        Afficher ligne
        Fermer lesEtudiants      //ferme le flux de données
    Sinon
        Afficher "Le fichier n'a pas pu être ouvert"
    FinSi
Fin
```

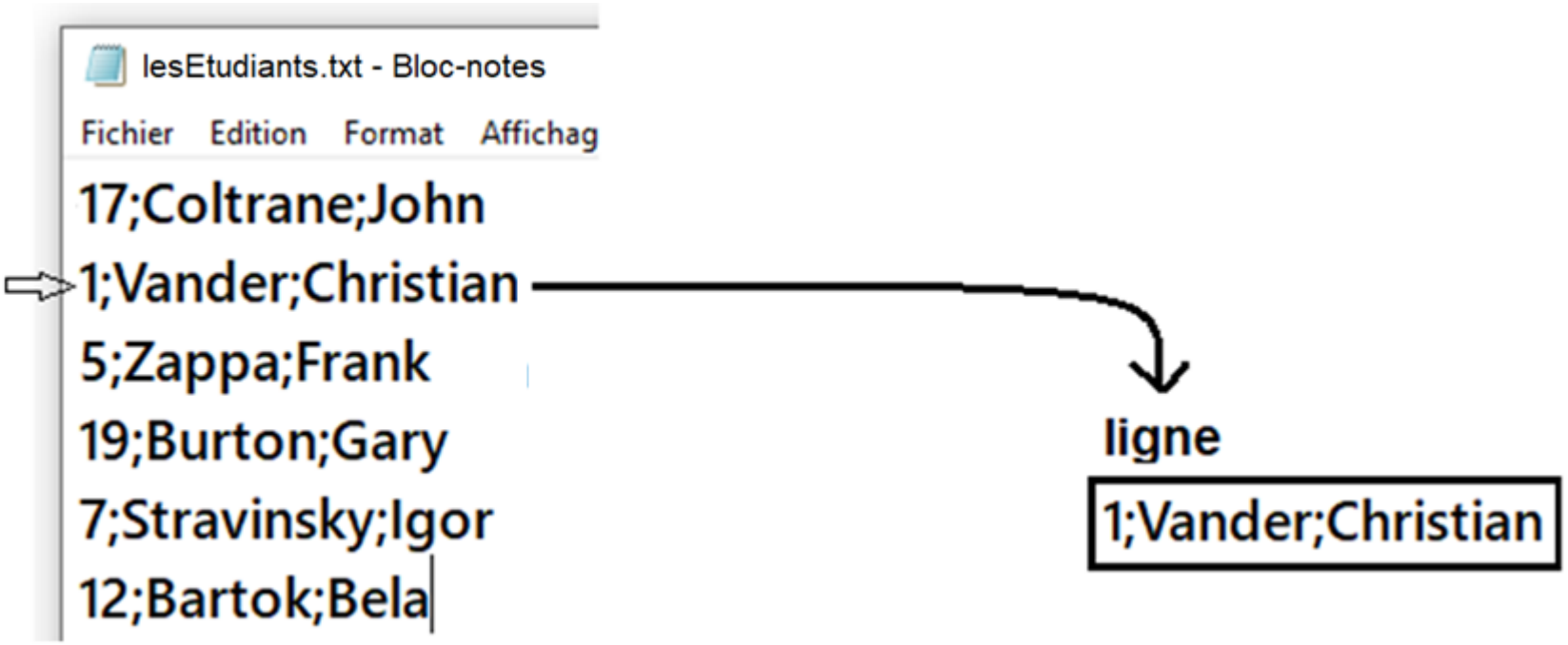
Lire le contenu du fichier

```
//1ère lecture
ligne = Lire lesEtudiants
```



Lire le contenu du fichier

```
//2ème lecture
ligne = Lire lesEtudiants
```



Lire le contenu du fichier

Aparté :

Après avoir lu le prochain enregistrement :

ligne = Lire lesEtudiants

J'obtiens :

ligne

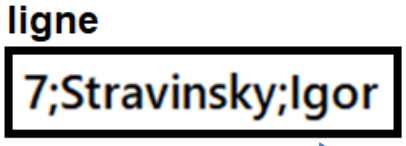
7;Stravinsky;Igor

Question : Comment récupérer chaque information contenue dans la variable ligne séparément ?

Lire le contenu du fichier

Réponse : je peux utiliser la fonction **diviser()** qui va diviser (éclater) la chaîne autour d'un séparateur (ou délimiteur) et remplir automatiquement un tableau de chaine. Par exemple :

```
ligne = Lire lesEtudiants
```



Ici le séparateur est ';' (';' ou ',' pour le format CSV)

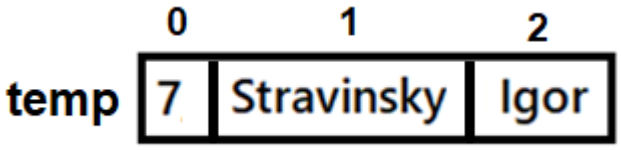
1ère étape - déclarer un tableau de chaines :

```
Variables
    temp : tableau de chaine
```

2ème étape - diviser la chaîne :

```
temp = diviser(ligne, ';')
```

Et j'obtiens :



Lire le contenu du fichier

Si je veux lire la totalité du fichier, j'ai besoin de savoir s'il reste des enregistrements à lire. J'utilise pour cela la fonction **finDeFichier()** qui revoie vrai ou faux :

```
Début
    Si Existe("lesEtudiants.txt") Alors
       Ouvrir lesEtudiants associé à "lesEtudiants.txt" en Lecture
       TantQue NON finDeFichier(lesEtudiants)
          ligne = Lire lesEtudiants
          Afficher ligne
       FinTantQue
       Fermer lesEtudiants
     Sinon
       Afficher "Le fichier n'a pas pu être ouvert"
    FinSi
```

Plan

1. Introduction sur les fichiers séquentiels
2. Mode d'exploitation d'un fichier séquentiel
3. Déclaration et ouverture d'un fichier
4. Lire le contenu du fichier
5. Écrire des enregistrements dans le fichier

Écrire des enregistrements dans le fichier

Si je veux écrire de nouveaux enregistrement, je dois ouvrir le fichier en mode **Ajout ou Ecriture** :

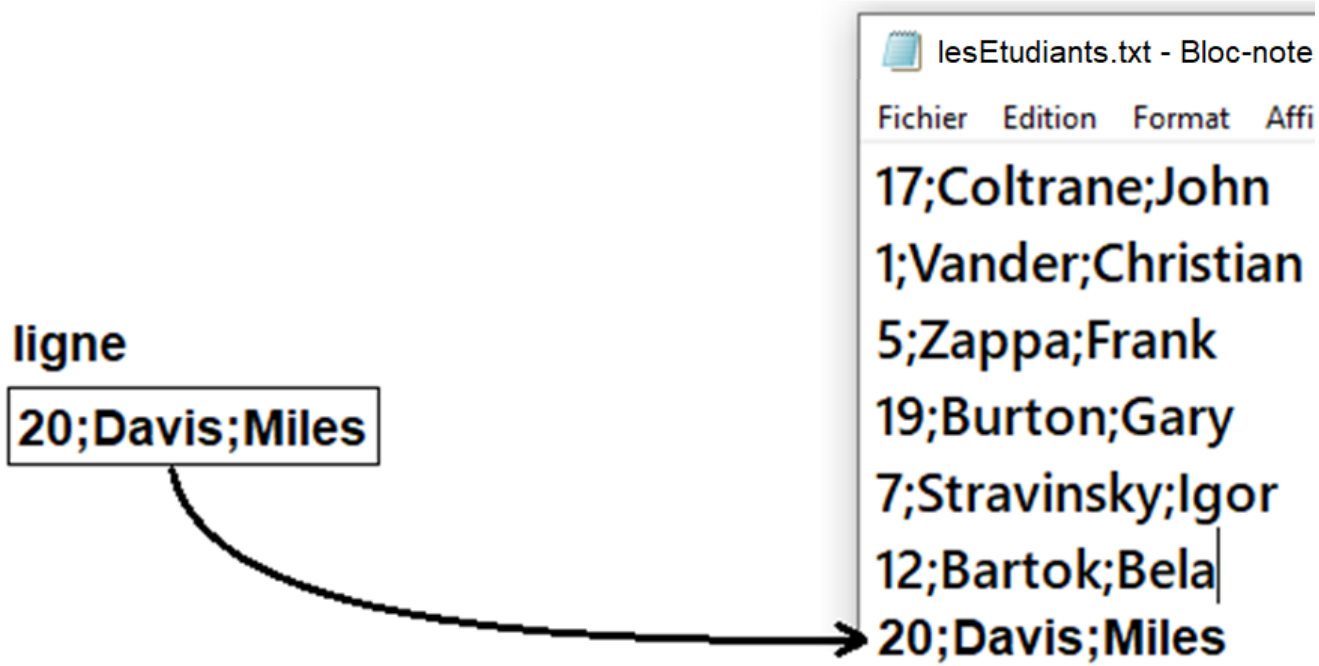
- En mode **ajout** les nouveaux enregistrements seront ajoutés à la fin du fichier
- En mode **Ecriture**, les nouveaux enregistrements seront ajoutés dans un fichier préalablement vidé.

```
Algorithme testFichier
    Déclarations
        Variables
            lesEtudiants : fichier séquentiel
            ligne : chaîne de caractères
Début
  Ouvrir lesEtudiants associé à "lesEtudiants.txt" en Ajout
  ligne = "20;Davis;Miles"
  Ecrire lesEtudiants, ligne
```

Écrire des enregistrements dans le fichier

Ecrire en mode Ajout

```
Si Existe("lesEtudiants.txt") Alors
    Ouvrir lesEtudiants associé à "lesEtudiants.txt" en Ajout
    ligne = "20;Davis;Miles"
    Ecrire lesEtudiants, ligne
```



Écrire des enregistrements dans le fichier

Ecrire en mode Ecriture

```
Si Existe("lesEtudiants.txt") Alors
    Ouvrir lesEtudiants associé à "lesEtudiants.txt" en Ecriture
    ligne = "20;Davis;Miles"
    Ecrire lesEtudiants, ligne
```



Les fichiers texte en C#

```
  //Nécessite de rajouter la bibliothèque System.IO;
using System.IO;

if(File.Exists("lesEtudiants.txt"))
{
      //Ouverture en Lecture
      //En 2 temps
      //1-on crée un flux qui permet de lire des fichiers…
      FileStream fs = new FileStream("lesEtudiants.txt",
            FileMode.Open, FileAccess.Read);
      //2-FileStream permet de lire des octets
      //pour préciser que l'on veut lire des caractères
      // il faut en plus déclarer un StreamReader
      StreamReader lesEtudiants = new StreamReader(fs);
}

   //Ouverture en mode Ajout
      FileStream fs = new FileStream("lesEtudiants.txt",
            FileMode.Append, FileAccess.Write);
      StreamWriter lesEtudiants = new StreamWriter(fs);


   //Ouverture en mode écriture (fichier vidé préalablement)
      FileStream fs = new FileStream("lesEtudiants.txt",
            FileMode.Create, FileAccess.Write);
      StreamWriter lesEtudiants = new StreamWriter(fs);
```

Les fichiers texte en C#

```
//Lire un enregistrement
    string ligne;
    ligne = lesEtudiants.ReadLine();


 //Lire tous les enregistrements
    while(!lesEtudiants.EndOfStream)
    {
        ligne = lesEtudiants.ReadLine();
    }

 //Ecrire un nouvel enregistrement
    ligne = "20;Davis;Miles";
    lesEtudiants.WriteLine(ligne);

 //Fermer le fichier
    lesEtudiants.Close();
```

Plan

1. Introduction sur les structures

Plan

1. Introduction sur les structures

Introduction sur les structures

Pour le moment nous avons travaillé uniquement avec les types de structure de données fournis de base :

- Types de base (entier, réel, booléen, …)
- Types construits prédéfinis (tableaux 1d et 2d)

qui peuvent s'avérer insuffisants pour représenter au mieux des situations du réel.

En algorithmique et dans la plupart des langages, le développeur à la possibilité de créer ses propres types : **les structures**.

Par exemple j'aimerais pouvoir réunir au sein d'une même structure les informations qui caractérisent un étudiant : numINE, nom, prénom, niveau, groupe TP et la moyenne des 6 semestres du BUT. Ce qui me permettrait de manipuler dans sa globalité un étudiant sans qu'il ait les informations qui le concernent éparpillées dans des variables différentes…

Plan

1. Introduction sur les structures
2. Création et déclaration en algorithmique

Création et déclaration en algorithmique

Voici comment créer et utiliser une structure :

```
Algorithme testStructure
    Structure Etudiant
        numINE : entier
        nom : chaîne
        prenom : chaîne
        niveau : chaîne
        groupeTP : entier
        resultats : tableau de 6 réels
    FinStructure
    Déclarations
        Variables
            unEtudiant : Etudiant
```

Je réunis ici toutes les informations qui caractérisent un étudiant

Attention : le nom de la structure s'écrit en PascalCase (majuscule à la 1ère lettre)

Plan

1. Introduction sur les structures
2. Création et déclaration en algorithmique
3. Manipulation d'une variable structurée

Manipulation d'une variable structurée

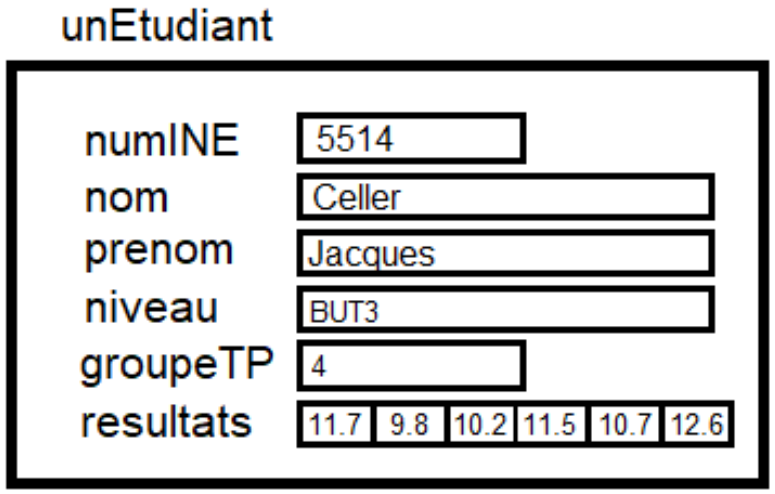
Voici comment créer une structure et initialiser une variable structurée :

```
Algorithme testStructure
    Structure Etudiant
        numINE : entier
        nom : chaîne
        prenom : chaîne
        niveau : chaîne
        groupeTP : entier
        resultats : tableau de 6 réels
    FinStructure
    Déclarations
        Variables
            unEtudiant : Etudiant
            somme : entier
            moyenne : réel
```

```
Début
    //Initialisation
    unEtudiant.numINE = 5514
    unEtudiant.nom = "Celler"
    unEtudiant.prenom = "Jacques"
    unEtudiant.niveau = "BUT3"
    unEtudiant.groupeTP = 4
    unEtudiant.resultat[0] = 11.7
    unEtudiant.resultat[1] = 9.8
    unEtudiant.resultat[2] = 10.2
    unEtudiant.resultat[3] = 11.5
    unEtudiant.resultat[4] = 10.7
    unEtudiant.resultat[5] = 12.6
```

Manipulation d'une variable structurée

Cet étudiant peut être représenté en mémoire comme ceci :



La variable unEtudiant contient directement les valeurs, il ne s'agit pas d'une référence.

On utilise une notation **pointée** pour atteindre les informations :

```
Afficher unEtudiant . numINE
```

nom de la variable

sous-partie de la structure

Manipulation d'une variable structurée

Je veux maintenant calculer et afficher la moyenne de cet étudiant :

```
Début
  //Initialisation
  unEtudiant.numINE = 5514
  unEtudiant.nom = "Celler"
  unEtudiant.prenom = "Jacques"
  unEtudiant.niveau = "BUT3"
  unEtudiant.groupeTP = 4
  unEtudiant.resultat[0] = 11.7
  unEtudiant.resultat[1] = 9.8
  unEtudiant.resultat[2] = 10.2
  unEtudiant.resultat[3] = 11.5
  unEtudiant.resultat[4] = 10.7
  unEtudiant.resultat[5] = 12.6

  //Calculer la moyenne
  somme = 0
  Pour i de 0 à 5
     somme = somme + unEtudiant.resultats[i]
  FinPour

  moyenne = somme / 6

   //Affichage du résultat
  Afficher "Etudiant : " + unEtudiant.numINE
  Afficher "Moyenne : " + moyenne
Fin
```

Plan

1. Introduction sur les structures
2. Création et déclaration en algorithmique
3. Manipulation d'une variable structurée
4. Vous avez dit "constructeur " ?

Vous avez dit "constructeur" ?

L'initialisation de cet étudiant vous parait fastidieuse ?

```
Début
    unEtudiant.numINE = 5514
    unEtudiant.nom = "Celler"
    unEtudiant.prenom = "Jacques"
    unEtudiant.niveau = "BUT3"
    unEtudiant.groupeTP = 4
    unEtudiant.resultat[0] = 11.7
    unEtudiant.resultat[1] = 9.8
    unEtudiant.resultat[2] = 10.2
    unEtudiant.resultat[3] = 11.5
    unEtudiant.resultat[4] = 10.7
    unEtudiant.resultat[5] = 12.6
```

on peut créer une procédure spéciale : un "**constructeur**" :

Vous avez dit "constructeur" ?

Cette procédure, au sein de la structure, qualifiée de "constructeur" porte tout simplement le nom de la structure et prend en paramètres les informations nécessaires à l'initialisation :

```
Structure Etudiant
    numINE : entier
    nom : chaîne
    prenom : chaîne
    niveau : chaîne
    groupeTP : entier
    resultats : tableau de 6 réels

    Procédure Etudiant(num : entier, nm :chaine, pnom : chaine, niv : chaine, grp : entier,
                       n1 : réel, n2 : réel, n3 : réel, n4 : réel, n5 : réel, n6 : réel)
    Début
        numINE = num            resultat[1] = n2
        nom = nm                resultat[2] = n3
        prenom = pnom           resultat[3] = n4
        niveau = niv            resultat[4] = n5
        groupeTP = gpr          resultat[5] = n6
        resultat[0] = n1
    Fin
FinStructure
```

Vous avez dit "constructeur" ?

Voici comment initialiser un nouvel étudiant à l'aide du constructeur :

```
    …
    Déclarations
        Variable
            unEtudiant : Etudiant
Début
    //Initialisation d'un étudiant avec son constructeur
    unEtudiant = new Etudiant("Celler","Jacques","BUT3",
                    4,11.7,9.8,10.2, 11.5,10.7,12.6)
Fin
```

Plan

1. Introduction sur les structures
2. Création et déclaration en algorithmique
3. Manipulation d'une variable structurée
4. Vous avez dit "constructeur " ?
5. Réunir plusieurs variables structurées

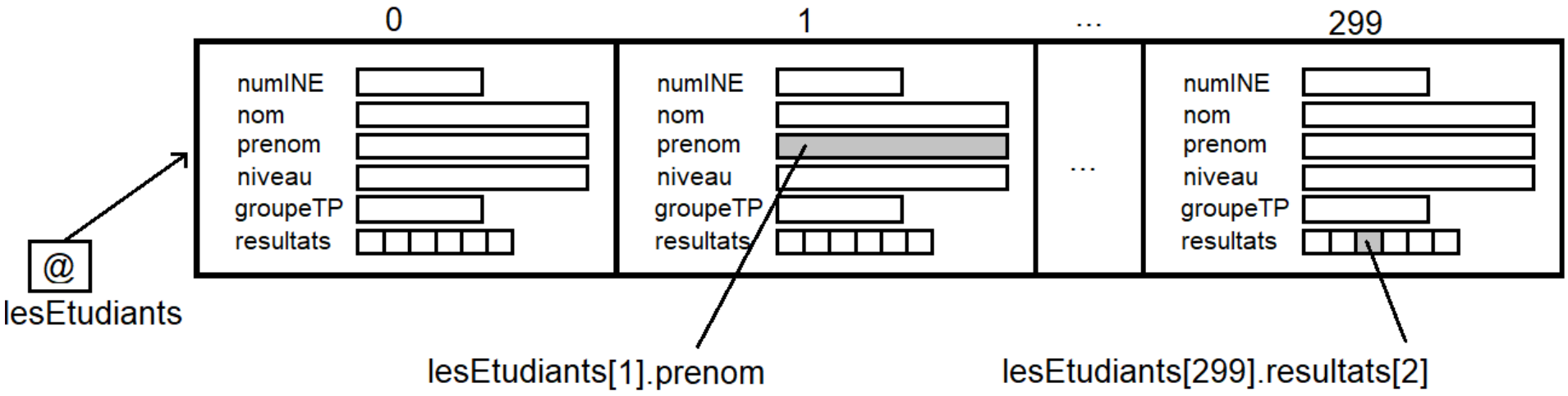
Réunir plusieurs variables structurées

On utilise tout simplement **un tableau d'étudiants**, mais prochainement nous verrons d'autres types qui permettront également de gérer des collections (liste, file, pile, dictionnaire) :

```
Algorithme testStructure
    Structure Etudiant
        numINE : entier
        nom : chaîne
        prenom : chaîne
        niveau : chaîne
        groupeTP : entier
        resultats : tableau de 6 réels
    FinStructure
    Déclarations
        Variables
            lesEtudiants : tableau de 300 Etudiants
```

Réunir plusieurs variables structurées

Le tableau **lesEtudiants** peut se représenter comme ceci en mémoire :



Attention, lesEtudiants est un tableau à 1 dimension, chaque case étant elle-même structurée. A ne pas confondre avec un tableau à 2 dimensions !

Réunir plusieurs variables structurées

Calculons maintenant la moyenne des BUT3 (je considère que le tableau lesEtudiants est déjà rempli) :

```
somme = 0
nbEtud = 0
Pour i de 0 à 299
    Si lesEtudiants[i].niveau  ==  "BUT3" Alors
        Pour j de 0 à 5
            somme = somme + lesEtudiants[i].resultats[j]
        FinPour
        nbEtud = nbEtud + 1
    FinSi
FinPour
moyenne = somme / nbEtud
Afficher "La moyenne des BUT3  = " + moyenne
```

Les structures en C#

```
class TestStruct
{
      struct Etudiant
      {
            public string nom;
            public string prenom;
            public int groupe;
            public float[] resultats;

            //Constructeur
            public Etudiant(string nm, string pn, int grp)
            {
                  nom = nm;
                  prenom = pn;
                  groupe = grp;
                  //Construction du tableau ici !
                  resultats = new float[6];
            }
      }

      public static void Main()
      {
            //Déclaration
            Etudiant et;
            //Création d'un nouvel étudiant
            et = new Etudiant("Zappa", "Frank", 3);
            Console.WriteLine(et.prenom + " " + et.nom);
```

//Obligatoire

Les structures

Le choix de créer les bonnes structures va désormais être un élément crucial de performance de vos algorithmes. Le but est de représenter au plus près les objets du monde réel, les structures étant un premier pas vers la programmation orientée objet (POO) que vous étudierez au seconde semestre.

Dans la SAÉ 11, vous êtes libres de créer les structures de votre choix, qui seront plus ou moins pertinentes…